

University of Pisa

Department of Information Engineering

Master's Degree in Electronic Engineering
Microelectronic System Design

Implementation of a Neural Network for Handwritten Character Recognition Using Altera MAX 10 FPGA

Academic Year 2025/2026

Students:
Dario Aliberti
Tommaso Tonarini

Contents

1 Introduction **2**
- 1.1 Altera DE10-lite dev. board . . . 2
- 1.2 Overview of the LCD Display Used . . . 3
- 1.3 Fully connected Neural Network . . . 4
  - 1.3.1 Neuron structure . . . 5
  - 1.3.2 Inference Process . . . 6
  - 1.3.3 Training Process . . . 7

2 Hardware implementation **8**
- 2.1 NiosII CPU modules description . . . 8
  - 2.1.1 Clock Modules . . . 8
  - 2.1.2 Memory Modules . . . 9
  - 2.1.3 NiosII Modules . . . 9
  - 2.1.4 PIO Modules . . . 10
  - 2.1.5 System peripherals . . . 10
  - 2.1.6 LT24 modules . . . 11
  - 2.1.7 Floating-point operator module . . . 11
- 2.2 Timing analysis . . . 12
  - 2.2.1 Timing configuration file . . . 12
  - 2.2.2 Maximum Frequency Report . . . 15
  - 2.2.3 Hold and Setup Time Slack Report . . . 16
- 2.3 Resource Usage . . . 17

3 Software implementation **18**
- 3.1 Neural Network implementation . . . 18
  - 3.1.1 Matrix Library . . . 19
  - 3.1.2 Neural Network Library . . . 21
- 3.2 GUI implementation . . . 24
  - 3.2.1 LCD Display Control Library . . . 29
  - 3.2.2 SPI-Based Touchscreen Control Library . . . 30
- 3.3 Main structure . . . 30

4 Debug and Verification **32**
- 4.1 Debug print methodology . . . 32
  - 4.1.1 Python Matrix visualization . . . 33
- 4.2 Python Neural Network confront . . . 33
- 4.3 Neural Network accuracy . . . 34
- 4.4 Neural Network inference time . . . 36
- 4.5 Project Demonstration . . . 37

5 Conclusions **38**
- 5.1 Possible upgrades . . . 38
- 5.2 Implementation on Different FPGA Devices . . . 39

1 Introduction

In this report, the implementation of a neural network capable of recognizing human handwriting on an FPGA will be analyzed.
To achieve this, the FPGA is connected to a touchscreen LCD display, where the user can write.
The hardware components used for this project are:

- **DE10-lite dev. board**:
  Is a development board, produced by terasIC that contains an Altera MAX10 FPGA and other useful peripherals like: button, switch, seven-segment displays, SDRAM and flash memory, etc.
- **LT24 Touch LCD Display**:
  Is a daughter board produced by terasIC that connects with the GPIO pins of the **DE10-lite** board and allow the communication between the FPGA and the LT24 LCD display.

1.1 Altera DE10-lite dev. board

The DE10-Lite is a low-cost FPGA development board based on the Intel (Altera) MAX10 device.
It is designed for system-level prototyping and is suitable for applications in the industrial, automotive, and consumer fields.
It provides a range of useful peripherals:

- USB-Blaster interface
- 64 MB SDRAM
- On-board flash memory
- Accelerometer
- Ambient light sensor
- VGA output interface
- GPIO header
- 7-segment display
- User LEDs
- Slide switches
- Push buttons
- On-board clock oscillator

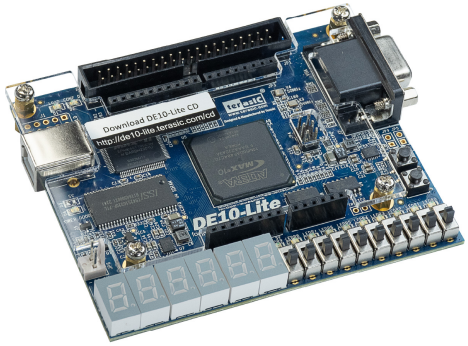


Figure 1: DE10 lite dev. board

1.2 Overview of the LCD Display Used

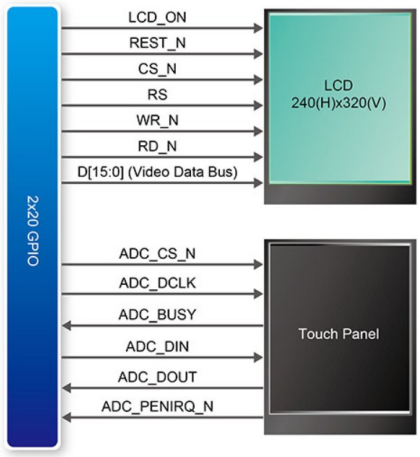


Figure 2: LT24 LCD Display block diagram

The LT24 LCD module integrates both a display panel and a resistive touch screen, which are interfaced using two different communication protocols. The touch panel uses an SPI interface to communicate with the AD7843 ADC integrated inside the module.
This device converts the analog touch coordinates into digital values, which are then read by the FPGA through the SPI bus.
The LCD panel is driven through a parallel bus interface, which is used to transmit pixel data for image rendering.
This interface allows the FPGA to directly write image data to the display for real-time visualization.

1.3 Fully connected Neural Network

A fully connected neural network is a structure composed of basic elements called **neurons**, which are interconnected.

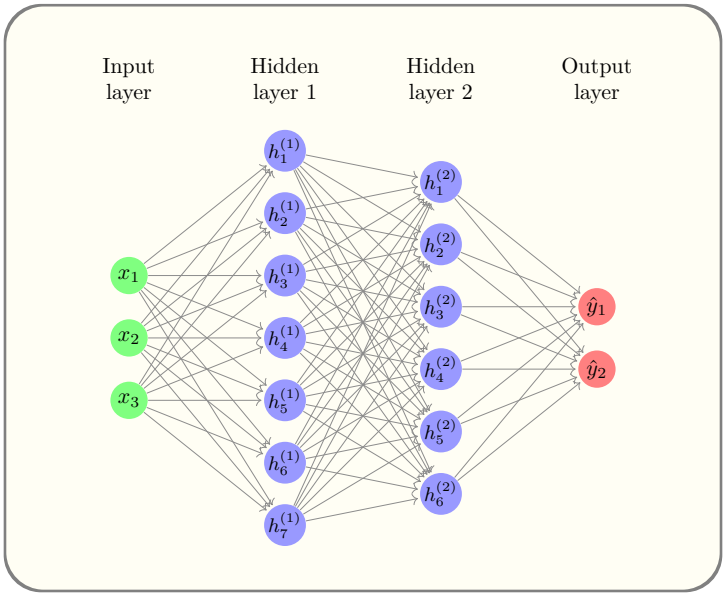


Figure 3: Fully connected Deep Neural Network with two hidden layers

As shown in Figure 3, neurons are arranged in different layers; the output of the neurons in one layer becomes the input to the neurons in the next layer.
The first layer is called the **input layer**, and its inputs correspond to the inputs of the neural network to be processed.
The last layer is called the **output layer**, and its output represents the final processed output of the neural network.
The intermediate layers are called **hidden layers** and may not be present in some networks.
Networks with at least one hidden layer are referred to as deep neural networks.

1.3.1 Neuron structure

To understand the behavior of a neural network, we must analyze the structure of a single neuron.

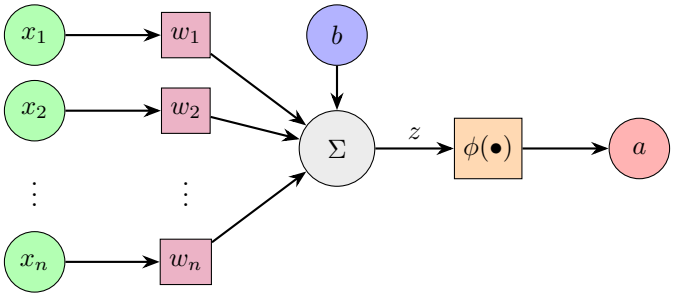


Figure 4: Neuron block diagram structure

As we can see in Figure 4, a neuron is a basic computational unit that processes input data to produce an output following the eq. (1)

$$\begin{cases} z_j &= \sum_i w_{ij} \cdot x_i + b_j \\ a_j &= \phi(z_j) \end{cases} \tag{1}$$

where:

- w_{ij}: Weight of the j-th neuron associated to the i-th input
- x_i: i-th input of the j-th neuron
- b_j: bias of the j-th neuron
- z_j: weighted sum of the j-th neuron
- $\phi(\bullet)$: activation function of the j-th neuron

The goal of using a neural network is to choose the correct values for the weights and biases of all neurons in order to obtain the desired output.
The process of choosing the weights and biases of the neurons is called the **Training process**.
The process of feeding inputs into the network to obtain its output is called the **Inference process**.

1.3.2 Inference Process

For a neural network, the inference process consist on computing the output of every neuron and give them as input of the neuron of the next layer of the network.
To simplify the computation process the inputs and the outputs are stored in a vector and the weights are stored into matrix like is shown in eq. (2).

$$\begin{bmatrix} z_1 & \cdots & z_n \end{bmatrix} = \begin{bmatrix} x_1 & \cdots & x_m \end{bmatrix} \begin{bmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m1} & \cdots & w_{mn} \end{bmatrix} + \begin{bmatrix} b_1 & \cdots & b_n \end{bmatrix} \tag{2}$$

where:

- n: is the number of outputs of the network
- m: is the number of inputs of the network

To obtain the output of a neuron layer, the weighted sum vector must pass through the activation function, like is shown in eq. (3).

$$\begin{bmatrix} a_1 & \cdots & a_n \end{bmatrix} = \begin{bmatrix} \phi(z_1) & \cdots & \phi(z_n) \end{bmatrix} \tag{3}$$

The activation functions can be various, but the most common ones are shown in Figure 5

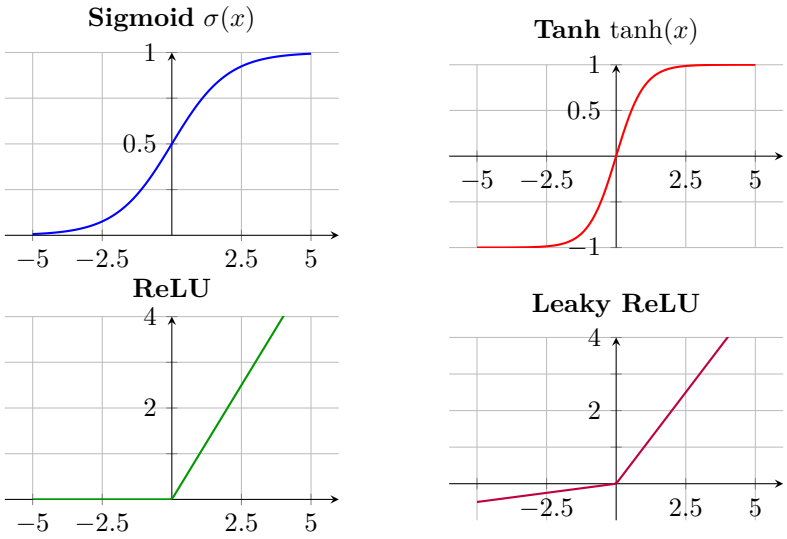


Figure 5: Most common activation functions

1.3.3 Training Process

During the training process, we need to send some inputs to the network and compare the obtained outputs with the desired ones.
Then, the weights and biases of the network must be modified to get closer to the desired output.
To train the network, an error function called the **cost function** is defined. This function represents the difference between the expected output of the network and the actual output, depending on the weights and biases of the network.
The goal of the training process is to choose the values of the weights and biases that minimize the cost function.
There are different types of cost functions, but the most common one is the quadratic cost function, defined in eq. (4)

$$C(\vec{w}, \vec{b}) = \frac{||\vec{y} - \vec{a}_{\text{out layer}}||^2}{2} \tag{4}$$

where:

- $\vec{w}$: is the weight vector
- $\vec{b}$: is the bias vector
- $\vec{a}_{\text{out layer}}$: is the output vector
- $\vec{y}$: is the desired output vector

If we consider small changes in the weights and biases, we can approximate the variation of the cost function as

$$\Delta C \sim \vec{\nabla} C \cdot \Delta(\vec{w}, \vec{b})$$

where

$$\vec{\nabla} C = \left(\frac{\partial C}{\partial w_1}, \cdots, \frac{\partial C}{\partial w_{m \cdot n}}, \frac{\partial C}{\partial b_1}, \cdots, \frac{\partial C}{\partial b_n} \right)$$

We start with random values for the weights and biases of the network.
Then, each time we send an input to the network, we compute the gradient of the cost function and update the weights and biases to reduce the value of the cost function.
If we update the weight and bias vectors as shown in eq. (5)

$$\Delta(\vec{w}, \vec{b}) = -\eta \vec{\nabla} C \tag{5}$$

we obtain an always negative variation of the cost function (according to the approximation above) until we reach its minimum value. In fact,

$$\Delta C \sim \vec{\nabla} C \cdot \Delta(\vec{w}, \vec{b}) = -\eta ||\vec{\nabla} C||^2 < 0$$

There are various types of cost functions, but the important property is that every function used must have a minimum when $\vec{y} = \vec{a}_{\text{out layer}}$.
It is easy to verify that the quadratic cost function satisfies this property, since it is a paraboloid.

2 Hardware implementation

2.1 NiosII CPU modules description

The CPU hardware structure is made of different modules, and every module have his scope.
The main modules are:

- **Clock modules**: Used to manage the different clock sources.
- **Memory modules**: Used to connect the CPU to different memories, either internal or external to the chip.
- **Nios II modules**: Used to instantiate the CPU module and the system ID module.
- **PIO modules**: Used to instantiate the I/O peripherals.
- **System peripherals**: Peripherals useful for programming and debugging, such as timers or UART devices.
- **LT24 modules**: Different modules used to control the LT24 LCD and touch screen (see [2]).
- **Floating-point operator module**: Used to instantiate hardware multipliers, dividers, and other floating-point operators.

2.1.1 Clock Modules

The clock section contains the modules required to generate and distribute the system clocks.
The `clk_in` module provides the external reference clock, while the `system_pll` module generates the internal clocks required by the system.
The `mm_clock_crossing_bridge` module is used to connect components working on different clock domains.

⊟ **clk_in**	Clock Source				
clk_in	Clock Input	**clk**	***exported***		
clk_in_reset	Reset Input	**reset**			
clk	Clock Output	*Double-click to export*	clk_in		
clk_reset	Reset Output	*Double-click to export*			
⊟ **system_pll**	ALTPLL Intel FPGA IP				
inclk_interface	Clock Input	*Double-click to export*	**clk_in**		
inclk_interface_reset	Reset Input	*Double-click to export*	[inclk_inte...		
pll_slave	Avalon Memory Mapped Slave	*Double-click to export*	[inclk_inte...	0x0900_9000	0x0900_900f
c0	Clock Output	*Double-click to export*	system_pll...		
c1	Clock Output	**clk_sdram**	system_pll...		
c2	Clock Output	*Double-click to export*	system_pll...		
c3	Clock Output	*Double-click to export*	system_pll...		
c4	Clock Output	**clk_vga**	system_pll...		
⊟ **mm_clock_crossi...**	Avalon-MM Clock Crossing Bridge				
m0_clk	Clock Input	*Double-click to export*	**system_...**		
m0_reset	Reset Input	*Double-click to export*	[m0_clk]		
s0_clk	Clock Input	*Double-click to export*	**system_...**		
s0_reset	Reset Input	*Double-click to export*	[s0_clk]		
s0	Avalon Memory Mapped Slave	*Double-click to export*	[s0_clk]	0x0800_0000	0x0800_03ff
m0	Avalon Memory Mapped Master	*Double-click to export*	[m0_clk]		

Figure 6: CPU clock modules instantiated

2.1.2 Memory Modules

The memory section contains the modules used for data and program storage. The `onchip_memory` module provides a small and fast internal memory, while the `sdram` module instantiate a controller for the external SDRAM memory to increase the available storage capacity.

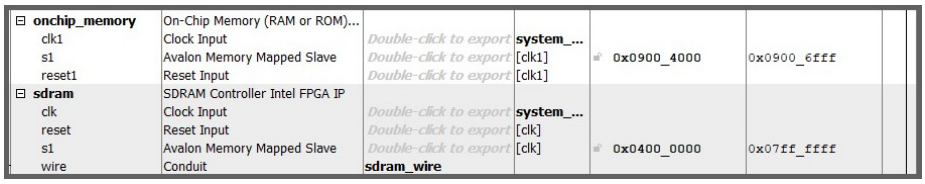


Figure 7: CPU memory modules instantiated

2.1.3 NiosII Modules

The main processing section is based on the `nios2_gen2` soft-core processor, which executes the software running on the FPGA.
The `sysid_qsys` module is used to identify the generated system configuration and ensure software compatibility.

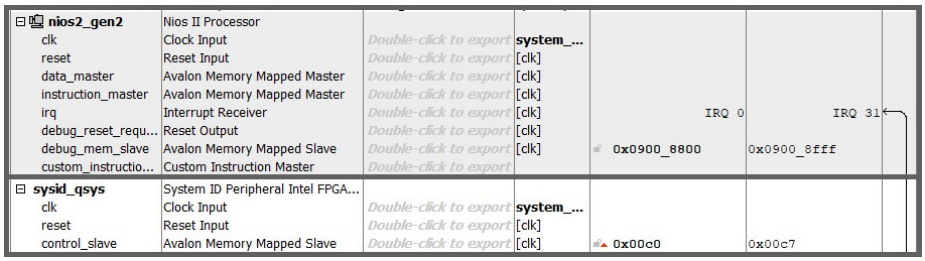
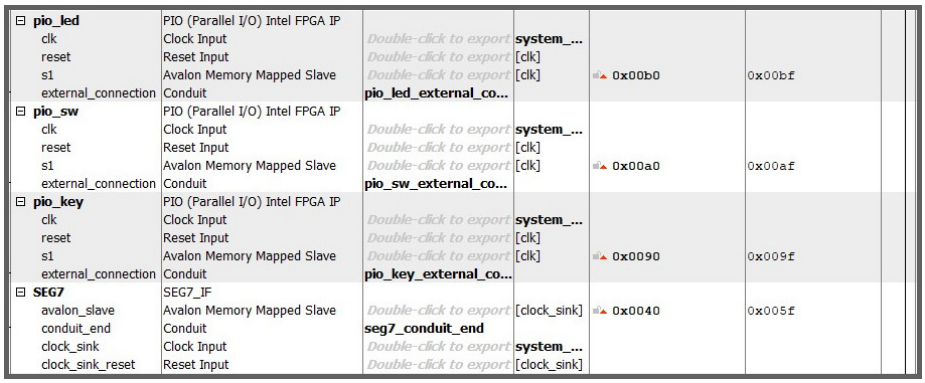


Figure 8: CPU (NiosII) modules instantiated

2.1.4 PIO Modules

The PIO modules are used to interface simple digital peripherals with the processor.
These modules manage input and output devices such as LEDs, switches, buttons, and other GPIO peripherals through memory-mapped registers.

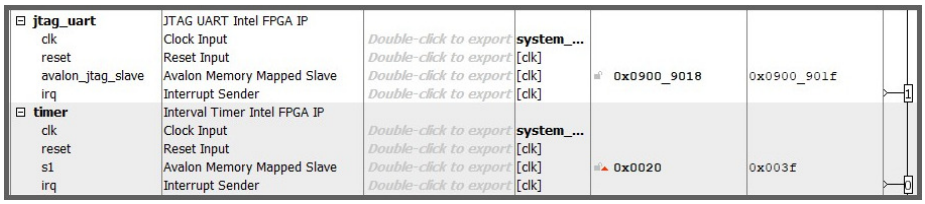


pio_led	PIO (Parallel I/O) Intel FPGA IP				
clk	Clock Input	*Double-click to export*	**system_...**		
reset	Reset Input	*Double-click to export*	[clk]		
s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x00b0	0x00bf
external_connection	Conduit	**pio_led_external_co...**			
pio_sw	PIO (Parallel I/O) Intel FPGA IP				
clk	Clock Input	*Double-click to export*	**system_...**		
reset	Reset Input	*Double-click to export*	[clk]		
s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x00a0	0x00af
external_connection	Conduit	**pio_sw_external_co...**			
pio_key	PIO (Parallel I/O) Intel FPGA IP				
clk	Clock Input	*Double-click to export*	**system_...**		
reset	Reset Input	*Double-click to export*	[clk]		
s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0090	0x009f
external_connection	Conduit	**pio_key_external_co...**			
SEG7	SEG7_IF				
avalon_slave	Avalon Memory Mapped Slave	*Double-click to export*	[clock_sink]	0x0040	0x005f
conduit_end	Conduit	**seg7_conduit_end**			
clock_sink	Clock Input	*Double-click to export*	**system_...**		
clock_sink_reset	Reset Input	*Double-click to export*	[clock_sink]		

Figure 9: CPU PIO modules instantiated

2.1.5 System peripherals

The system peripheral section contains modules used for communication and timing operations.
The `jtag_uart` module allows serial communication and debugging through the JTAG interface, while the `timer` module provides timing and delay functionalities for the software running on the processor.



jtag_uart	JTAG UART Intel FPGA IP				
clk	Clock Input	*Double-click to export*	**system_...**		
reset	Reset Input	*Double-click to export*	[clk]		
avalon_jtag_slave	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0900_9018	0x0900_901f
irq	Interrupt Sender	*Double-click to export*	[clk]		
timer	Interval Timer Intel FPGA IP				
clk	Clock Input	*Double-click to export*	**system_...**		
reset	Reset Input	*Double-click to export*	[clk]		
s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0020	0x003f
irq	Interrupt Sender	*Double-click to export*	[clk]		

Figure 10: CPU system peripherals instantiated

2.1.6 LT24 modules

The LT24 section contains the modules required to control the LCD touchscreen display.
The `LT24_Controller` module manages the communication with the display, while `LCD_RESET_N` handles the display reset signal.
The touchscreen interface is managed through the `touch_panel_spi`, `touch_panel_pen_irq`, and `touch_panel_busy` modules.

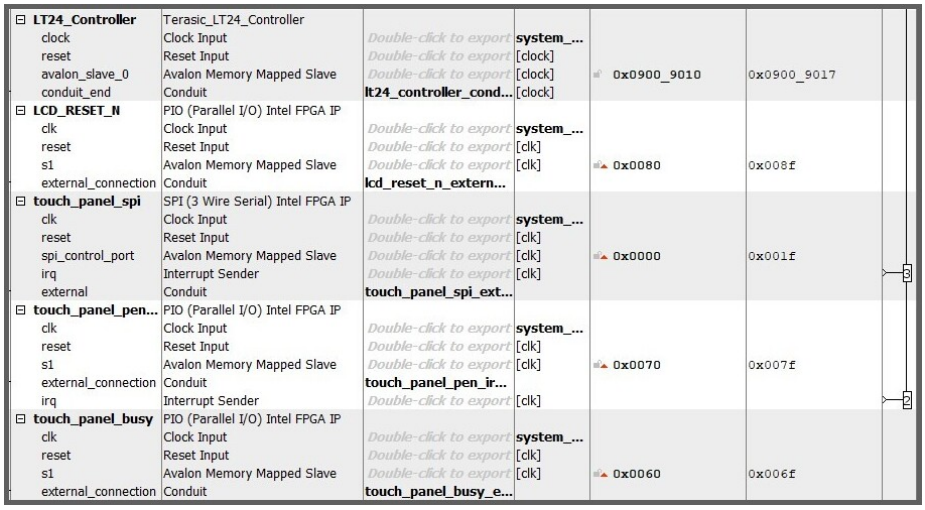


Figure 11: CPU LT24 modules instantiated

2.1.7 Floating-point operator module

The floating-point operator module provides hardware support for floating-point arithmetic operations.



Figure 12: CPU floating-point operator module instantiated

2.2 Timing analysis

This section defines the physical root clocks entering the device and characterizes the internal clock trees synthesized by the Phase-Locked Loop (PLL) hardware macro.
These constraints ensure that the Static Timing Analysis engine can accurately verify setup and hold timing margins across all clock domains.

2.2.1 Timing configuration file

The provided Synopsys Design Constraints (SDC) code snippet establishes the fundamental timing environment for the FPGA design.

```
    create_clock
        -name {altera_reserved_tck}
        -period 100.000
        -waveform { 0.000 50.000 }
        [get_ports {altera_reserved_tck}]
    create_clock
        -name {ADC_CLK_10}
        -period 100.000
        -waveform { 0.000 50.000 }
        [get_ports {ADC_CLK_10}]
    create_clock
        -name {MAX10_CLK1_50}
        -period 20.000
        -waveform { 0.000 10.000 }
        [get_ports {MAX10_CLK1_50}]

    derive_pll_clocks
```

Listing 1: Base clock settings for timing analyzer

The base clocks settings present in Listing 1 represent physical, periodic signals routed from external oscillators on the board to specific input ports of the FPGA.

- `altera_reserved_tck`
  is dedicated to the asynchronous JTAG Test Clock tap for hardware debugging.
- `ADC_CLK_10`
  is the 10 MHz clock used to drive the controller of the LT24 touchscreen.
- `MAX10_CLK1_50`
  these serve as the primary external reference clocks feeding the internal logic and PLL inputs.
- `derive_pll_clocks`
  this command instructs Intel Quartus Prime to automatically detect internal PLL instances.

Generated clocks represent signals derived from an existing master clock inside the FPGA, altered via frequency multiplication, division, or phase shifts.

```
    create_generated_clock
        -name {clk_dram_ext}
        -source [get_pins{ u0|system_pll|sd1|pll7|clk[1] }]
        [get_ports {DRAM_CLK}]
    create_generated_clock
        -name {u0|system_pll|sd1|pll7|clk[0]}
        -source [get_pins{u0|system_pll|sd1|pll7|inclk[0]}]
        -duty_cycle 50/1 -multiply_by 2
        -master_clock {MAX10_CLK1_50}
        [get_pins {u0|system_pll|sd1|pll7|clk[0]}]
    create_generated_clock
        -name {u0|system_pll|sd1|pll7|clk[3]}
        -source [get_pins{u0|system_pll|sd1|pll7|inclk[0]}]
        -duty_cycle 50/1 -multiply_by 1
        -divide_by 5 -master_clock {MAX10_CLK1_50}
        [get_pins{u0|system_pll|sd1|pll7|clk[3]}]
```

Listing 2: Generated clock settings for timing analyzer

The paths originating from the switches and push-buttons were subsequently defined as asynchronous (unconstrained), while timing constraints were also specified for the paths driving the seven-segment displays and leds.

```
    set_false_path
        -from [get_ports {SW[*] KEY[*]}]
    set_false_path
        -from [get_ports {LT24_ADC_*}]
    set_false_path
        -to [get_ports {HEX0[*] HEX1[*] HEX2[*] HEX3[*]
                        HEX4[*] HEX5[*] LEDR[*]}]
```

Listing 3: Paths constraints for asynchronous buttons and leds on the board

Furthermore, based on the timing specifications provided in the corresponding datasheets, minimum and maximum output delays were defined for both the SDRAM interface and the JTAG Debug Module (USB Blaster).
The timing constraints for the JTAG Debug Module (USB Blaster) were derived using the forum [7].
For the LT24 display interface, since it is not directly synchronized to the system clock and all timing requirements are handled internally by a controller operating at 10 MHz, arbitrary minimum and maximum output delay values (1 ns in this implementation) were assigned relative to the clock signal.
In this case, verifying synchronization with the clock is sufficient to guarantee correct timing behavior, as the LT24 controller itself enforces the required interface timing.

```
    set_input_delay
        -add_delay -max -clock [get_clocks {clk_dram_ext}]
        5.900 [get_ports {DRAM_DQ[*]}]
    set_input_delay
        -add_delay -min -clock [get_clocks {clk_dram_ext}]
        3.000 [get_ports {DRAM_DQ[*]}]
    set_output_delay
        -add_delay -max -clock [get_clocks {clk_dram_ext}]
        1.600 [get_ports {DRAM_ADDR[*]}]
    set_output_delay
        -add_delay -min -clock [get_clocks {clk_dram_ext}]
        -0.900 [get_ports {DRAM_ADDR[*]}]
```

Listing 4: Paths constraints for SDRAM

```
    set_output_delay
        -add_delay
        -clock [get_clocks {u0|system_pll|sd1|pll7|clk[3]}]
        1.000 [get_ports {LT24_*}]
```

Listing 5: Paths constraints for LT24 module

```
    set_input_delay
        -add_delay -max
        -clock [get_clocks {altera_reserved_tck}]
        10.000 [get_ports {altera_reserved_tdi}]
    set_input_delay
        -add_delay -min
        -clock [get_clocks {altera_reserved_tck}]
        2.000 [get_ports {altera_reserved_tdi}]
    set_input_delay
        -add_delay -max
        -clock [get_clocks {altera_reserved_tck}]
        10.000 [get_ports {altera_reserved_tms}]
    set_input_delay
        -add_delay -min
        -clock [get_clocks {altera_reserved_tck}]
        2.000 [get_ports {altera_reserved_tms}]
    set_output_delay
        -add_delay -max
        -clock [get_clocks {altera_reserved_tck}]
        10.000 [get_ports {altera_reserved_tdo}]
    set_output_delay
        -add_delay -min
        -clock [get_clocks {altera_reserved_tck}]
        2.000 [get_ports {altera_reserved_tdo}]
```

Listing 6: Paths constraints for JTAG Debug Module (USB Blaster)

2.2.2 Maximum Frequency Report

The timing analyzer uses the library of the FPGA macro-cells to verify that the timing constraints are respected for both the slow and fast models.
The slow model is used to check the maximum frequency of the clock sources (at a temperature of $85°C$ worst case) and the setup timing constraints.
The fast model is used to verify that hold time violations do not occur (at a temperature of $0°C$ best case).

	Fmax	Restricted Fmax	Clock Name	Note
1	22.96 MHz	22.96 MHz	altera_reserved_tck	
2	73.73 MHz	73.73 MHz	u0|system_pll...1|pll7|clk[3]	
3	114.68 MHz	114.68 MHz	u0|system_pll...1|pll7|clk[0]	
4	322.27 MHz	250.0 MHz	MAX10_CLK1_50	limi...ate)

Figure 13: Maximum frequency Timing analyzer report

- `Internal System Clock (clk[0])`
  The Nios II CPU and all associated modules meet the timing requirements. Since the maximum achievable frequency (114.68 MHz) exceeds the target frequency (100 MHz), the system operates with positive timing slack.
- `LT24 Display Clock (clk[3])`
  Although the internal logic could theoretically operate up to 73.73 MHz, only 10 MHz is required for the LT24 interface. As specified in the LT24 datasheet, higher frequencies may cause data corruption; therefore, the clock frequency is kept at or below 10 MHz despite the available timing margin.
- `JTAG Debug Clock (altera_reserved_tck)`
  The timing analysis confirms compliance with the required 10 MHz operating frequency.

Note: The timing analyzer indicates that while the internal silicon routing from this clock pin is capable of supporting frequencies up to 322.27 *MHz, the physical I/O buffer architecture of the MAX 10 device imposes a hard limit at* 250.0 *MHz. This limitation is due to the electrical switching capability of the I/O transistors, which cannot reliably toggle beyond this frequency.*
However, this constraint is not a concern in practice, since the operating frequency of 50 *MHz is well below both the I/O limit (*250 *MHz) and the internal routing limit (*322.27 *MHz), resulting in a substantial safety margin for this clock domain. Quartus therefore caps the reported maximum frequency at* 250 *MHz to prevent unrealistic external clock assumptions in future design iterations.*

2.2.3 Hold and Setup Time Slack Report

The timing analyzer evaluates the setup and hold slack for each clock domain to ensure correct data latching and prevent metastability.
The setup analysis checks that the data arrives at the destination register before the next clock edge, whereas the hold analysis ensures the data remains stable long enough after the clock edge to be properly sampled.
As shown in the Figure 14, all clock domains exhibit positive slack, confirming that the design fully meets all timing requirements under both worst-case and best-case operating conditions.

	Clock	Slack	End Point TNS
1	u0|system_pll|sd1|pll7|clk[3]	0.307	0.000
2	u0|system_pll|sd1|pll7|clk[0]	0.308	0.000
3	altera_reserved_tck	0.324	0.000
4	MAX10_CLK1_50	0.328	0.000
5	clk_dram_ext	2.665	0.000

(a) Hold summary report

	Clock	Slack	End Point TNS
1	u0|system_pll|sd1|pll7|clk[0]	1.280	0.000
2	clk_dram_ext	3.566	0.000
3	MAX10_CLK1_50	16.897	0.000
4	altera_reserved_tck	28.227	0.000
5	u0|system_pll|sd1|pll7|clk[3]	43.795	0.000

(b) Setup summary report

Figure 14: Hold and Setup slack Timing analyzer reports

- `Internal System Clock (clk[0])`
  The system clock meets both timing constraints comfortably. It presents a worst-case hold slack of 0.308 ns and a setup slack of 1.280 ns. The positive margins ensure stable synchronous operations within the Nios II CPU and its peripherals.

- `LT24 Display Clock (clk[3])`
  The clock domain dedicated to the LT24 display interface reports a hold slack of 0.307 ns and a substantial setup slack of 43.795 ns. This significant setup margin is a direct consequence of the relaxed operating frequency (10 MHz) compared to the maximum frequency supported by the internal logic routing.

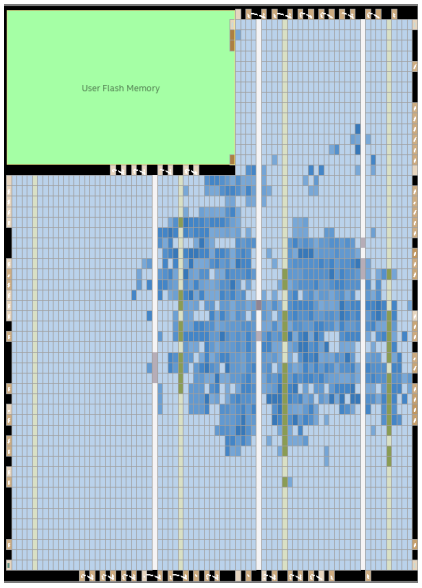
- `External DRAM Clock (clk_dram_ext)`
  The external memory clock domain exhibits high robust timing margins, showing a hold slack of 2.665 ns and a setup slack of 3.566 ns. This ensures reliable data transfer with the external SDRAM chip.

- `System Input Clock (MAX10_CLK1_50)`
  The 50 MHz main input clock shows a hold slack of 0.328 ns and a setup slack of 16.897 ns, confirming safe propagation of the external reference clock into the internal PLL structures.

- `JTAG Debug Clock (altera_reserved_tck)`
  The JTAG test clock complies with the constraints, reporting a hold slack of 0.324 ns and a setup margin of 28.227 ns.

2.3 Resource Usage

Compared to all the resources available on the chip, the total logic elements used are only 17% of the total, and the total memory bits used are 11%, as shown in Figure 15.
Furthermore only 16 of the 288 embedded multipliers and one of the four internal PLL.



(a) Hardware-Footprint on Chip Planner

Fitter Status	Successful - Mon May 25 09:25:02 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	NN_letter_recognition
Top-level Entity Name	NN_letter_recognition
Family	MAX 10
Device	10M50DAF484C6GES
Timing Models	Preliminary
Total logic elements	8,529 / 49,760 (17 %)
Total registers	4885
Total pins	177 / 360 (49 %)
Total virtual pins	0
Total memory bits	180,992 / 1,677,312 (11 %)
Embedded Multiplier 9-bit elements	16 / 288 (6 %)
Total PLLs	1 / 4 (25 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

(b) Fitter summary hardware used

Figure 15: FPGA resources usage

3 Software implementation

The software architecture is divided into two main parts.
Neural Network implementation is based on two different libraries:

- **Matrix Library**: responsible for matrix operations and mathematical computations required by the neural network.
- **Neural Network Library**: handles the initialization and execution of a fully connected neural network composed of three layers.

Graphical User Interface (GUI) implementation relies on two dedicated libraries used to manage communication with the display and the touchscreen:

- **LCD Display Control Library**: handles everything related to displaying images on the LCD screen.
- **SPI-Based Touchscreen Control Library**: manages communication with the touchscreen using the Serial Peripheral Interface (SPI).

3.1 Neural Network implementation

The Neural Network library folder tree is shown in Figure 16

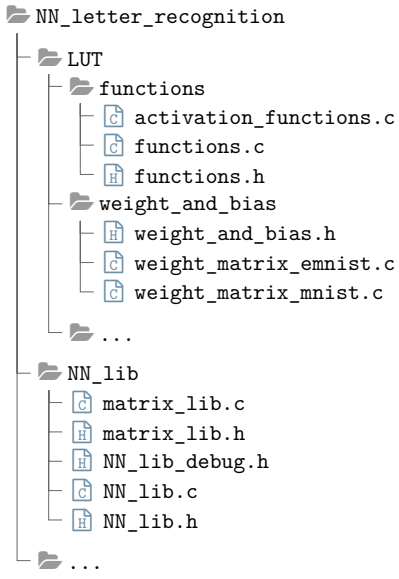


Figure 16: Neural Network library function folder tree

To implement the neural network, a matrix library has been created to define all the operations that have to be performed with matrices, and a neural network library has been created to define the network and connect all the layers together.

3.1.1 Matrix Library

This library contains the `Matrix` type, which is a structure defined in Listing 7.

```
typedef struct{
    unsigned int row;
    unsigned int col;
    float *data;
}Matrix;
```

Listing 7: Matrix type

The `Matrix` type contains the fields `row` and `col`, which store the number of rows and columns of the matrix, and the field `*data`, which is a pointer to the matrix data.
This type is defined to represent a matrix of floating-point values only, since the weights of the neural network are of type `float`.
To create a matrix variable, the `malloc()` function must be used to allocate memory in the heap.
In `matrix_lib.h`, several functions are defined to instantiate a matrix variable (constructors) and to release its memory (destructor) shown in Listing 8.

```
/* ---------------- Constructors ------------------ */
static inline Matrix matrix_error(void) {
    return (Matrix){0, 0, NULL};
}

Matrix matrix_init(unsigned int row, unsigned int col);
...
Matrix matrix_null(unsigned int row, unsigned int col);

/* ----------------- Destructors ------------------- */
void matrix_free(Matrix *m);
```

Listing 8: Constructors and Destructor

To instantiate a matrix variable the syntax is shown in Listing 9.

```
1   int main(void){
2       Matrix m = matrix_init(ROW_NUM, COL_NUM);
3
4       Matrix error_m = matrix_error();
5       if(matrix_compare(&error_m, &m) == 1){
6           printf("ERROR: Matrix initialization failure");
7           return 0;   // Exit failure
8       }
9
10      ...
11
12      matrix_free(&m);
13      return 1;   // Exit success
14  }
```

Listing 9: Matrix usage example

The `matrix_error(void)` constructor is used only to return an empty Matrix, useful to identify an error in a `Matrix` return type function as shown in Listing 9.
Since `matrix_error(void)` don't allocate data into dynamic memory (is an empty matrix) the destructor must not be used.
In this library, the most common matrix operations such as addition, multiplication, transposition, etc., have also been defined.
Most of the operations are implemented as functions that return an integer value representing the status; if the return value is 1, the operation has been completed successfully; otherwise, the operation has encountered a problem and cannot be executed correctly.

3.1.2 Neural Network Library

This library contains the `NN_f_conn` type, which is a structure defined in Listing 10

```
typedef struct{

    Matrix w1;
    Matrix b1;

    Matrix w2;
    Matrix b2;

    Matrix w3;
    Matrix b3;

}NN_f_conn;
```

Listing 10: NN_f_conn type

With this structure can be defined fully connected neural network with three layers:

- The input layer
- An hidden layer
- The output layer

This structure contains three pairs of matrices defined with the `Matrix` type described in subsubsection 3.1.1.
Each pair of matrices represents the weights and biases of a layer of the fully connected neural network.
The neural network is initialized using the function `NN_init()`, which initializes all the matrices with the correct dimensions.
Since the matrices contain data arrays allocated using the `malloc()` function, the `free()` function must be used to de-allocate them; the function that de-allocates all the network data is `NN_free()` as shown in Listing 11.

```
/* ---------------- Constructors ------------------ */
NN_f_conn NN_init(void);

/* ----------------- Destructors -------------------- */
void NN_free(NN_f_conn *nn);
```

Listing 11: Constructors and Destructor

The dimensions of the network layers are defined in another file located in the `./LUT/weight_and_bias` directory, named `weight_and_bias.h`.
In this file, a global `Matrix` vector is defined, which contains all the weights and biases of the network.

To identify at which index each matrix is stored, a global enumeration variable has been defined, as shown in Listing 12.

```
typedef enum{

    WEIGHT_NN_1 = 0,
    BIAS_NN_1 = 3,

    WEIGHT_NN_2 = 1,
    BIAS_NN_2 = 4,

    WEIGHT_NN_3 = 2,
    BIAS_NN_3 = 5,

}NN_weight_index;
```

Listing 12: Enumerate variable to identify the NN index

Since the network used for the MNIST dataset and the EMNIST dataset is, in both cases, a three-layer fully connected network (with a different number of neurons in each layer), conditional compilation is used in the header file to load one network or the other, as shown in Listing 13.

```
// Comment this line to use the EMNIST dataset
// (letters and numbers) instead of MNIST (only numbers)
#define MNIST_DATASET

#ifdef MNIST_DATASET
    #define WEIGHT_AND_BIAS weight_and_bias_mnist
    extern Matrix weight_and_bias_mnist[6];
#else
    #define WEIGHT_AND_BIAS weight_and_bias_emnist
    extern Matrix weight_and_bias_emnist[6];
#endif
```

Listing 13: NN selection with conditional compilation

`weight_and_bias_mnist[6]` and `weight_and_bias_emnist[6]` are two constant arrays of type `Matrix`, defined respectively in `./LUT/weight_and_bias/weight_and_bias_mnist.c` and `./LUT/weight_and_bias/weight_and_bias_emnist.c`.
These matrix vectors contain the weights and biases for the MNIST and EMNIST networks.

Inside `NN_lib`, the `NN_init()` function uses the `WEIGHT_AND_BIAS` define to access the constant matrices associated with the correct dataset.
The operation that can be performed are shown in Listing 14

```
1   /* ---------------- Operations -------------------- */
2   int forw_prop(Matrix input,
3                 Matrix *out,
4                 NN_f_conn *nn,
5                 void (*activation_funct)(Matrix *a));
6   char nn_output_convert(Matrix input);
7
8   /* ------------- Activation functions ------------- */
9   void relu_funct(Matrix *a);
10  void leaky_relu_funct(Matrix *a);
11  void tanh_funct(Matrix *a);
12  void sigmoid_funct(Matrix *a);
```

Listing 14: NN_lib functions defined

The `forw_prop()` function takes a `Matrix` type input and produces a `Matrix` output from the `NN_f_conn` neural network, using activation functions passed as function pointers.
Several activation functions are defined; the most computationally expensive ones are implemented using look-up tables stored in
`./LUT/functions/functions.h`.
The function `nn_output_convert` determines which neural network has been selected and returns the corresponding output character from the input matrix, where the input matrix is the result computed by the network.

3.2 GUI implementation

The GUI library folder tree is shown in Figure 17

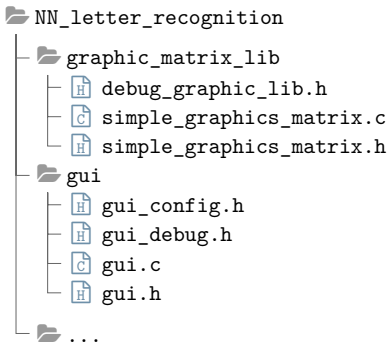


Figure 17: GUI library function folder tree

The GUI library handles the operations needed to render the user interface on the display and enables touchscreen sensitivity, allowing the user to handwrite their desired character.

```
1   void GUI_init(alt_video_display *pDisplay,
2                 DESK_INFO *DeskInfo,
3                 RECT *rcTouch)
4   {
5       // Init LCD
6       LCD_Init();
7
8       // Clean LCD
9       vid_clean_screen(pDisplay, BACKGROUND_COLOR);
10
11      // Wait 1s
12      usleep(1*1000*1000);
13
14      (*DeskInfo).Button_Index = PEN_BUTTON;
15
16      GUI_DeskInit(pDisplay, DeskInfo);
17      GUI_DeskDraw(pDisplay, DeskInfo);
18      GUI_SymbolsDraw(pDisplay, DeskInfo, 0);
19
20      RectCopy(rcTouch, &(*DeskInfo).rcPaint);
21      RectInflate(rcTouch, -2-DOT_SIZE, -2-DOT_SIZE);
22  }
```

Listing 15: GUI initialization

During initialization, the program builds and displays the interface (`GUI_init()`), through functions like `GUI_DeskInit()`, `GUI_DeskDraw()`, `GUI_SymbolsDraw()`.
Then (Listing 16) it then enters a while loop that continuously detects (an alarm is implemented) the pointer's position `Touch_GetXY()`, based on where the pen presses down. The task of checking whether the pointer is inside a specific section of the display is handled by the library function `IsPtInRect()`.

```
    if(Touch_GetXY(pTouch, &X, &Y)){

        PtSet(&Pt, X, Y);
        int ButtonId = GUI_CheckButton(DeskInfo, &Pt);

        GUI_DEBUG_TOUCH_PRINTF("x=%d, y=%d\r\n", X, Y);
        GUI_DEBUG_TOUCH_PRINTF("Button index=%d\n",
                               (*DeskInfo).Button_Index);

        if (IsPtInRect(&Pt, rcTouch) &&
            (*DeskInfo).Button_Index == PEN_BUTTON){
            // Touch inside drawing area
            if (Pt_prev.x != -1 && Pt_prev.y != -1)
                vid_draw_sloped_line_on_matrix( Pt_prev.x,
                Pt_prev.y, Pt.x, Pt.y, DOT_SIZE, BLACK_24,
                pDisplay, &drawing_grayscale_m);
            else
                vid_draw_circle_on_matrix(Pt.x, Pt.y,
                DOT_SIZE, BLACK_24, DO_FILL, pDisplay,
                &drawing_grayscale_m);

            PtSet(&Pt_prev, X, Y);
        }
    }
```

Listing 16: Handling the touchscreen

To draw the pen strokes, we used the `vid_draw_sloped_line_on_matrix()` function.
This "interpolates" the detected touchpoints by connecting them with a sloped line.
This approach gives us a very smooth drawing and writing experience without needing to bump up the touch-detection refresh rate, avoiding extra overhead.

On the other hand, if a press is detected on one of the available virtual buttons (or icons), the system behaves differently:

- `BACKSPACE_BUTTON`: deletes the last character written in the letter box at the top of the display. If the letter box fills up, the last entered character is overwritten.
- `PEN_BUTTON`: enables pen writing exclusively within the dedicated drawing area. There is a timer that triggers once the pen is lifted; when it times out, the data saved in the 140x140 matrix is downscaled to a 28x28 grayscale matrix, by the `downscale_matrix()` function, and fed into the neural network.
- `ERASE_BUTTON`: clears everything written in the letter box as well as any letter currently being drawn, resetting the interface to its initial state.

```
    if(ButtonId == BACKSPACE_BUTTON){

        GUI_ClearButtonArea(pDisplay, DeskInfo);
        GUI_SymbolsDraw(pDisplay, DeskInfo, PressedButton);
        Touch_EmptyFifo(pTouch);

        // Delete a letter
        letter_index--;
        if(letter_index < 0){
            letter_index = 0;
        }
        else{
            GUI_ClearSelectedLetter(pDisplay, DeskInfo,
            letter_index);
            GUI_DeskDraw(pDisplay, DeskInfo);
        }

    }
```

Listing 17: Handling the buttons

As an example, only the operations that are performed after the `BACKSPACE_BUTTON` is pressed are shown.
Once the pen is released for `WAIT_LETTER_TIME_MS` (which we set to 1.5 seconds in our tests), the cleanup operations kick off: the drawing area is cleared via `GUI_ClearPaintArea()`, and the touch SPI's FIFO memory is emptied.
After that, the matrix is downscaled and adapted using `downscale_matrix()` and `matrix_flatten()` so it's in the right format to be fed into the neural network for forward propagation `via forw_prop()` function.

```
    if (((t_wait_1 - t_wait_0) >= WAIT_LETTER_TIME_MS) &&
        (ActiveDrawing == 1)){

        GUI_ClearPaintArea(pDisplay, DeskInfo);
        GUI_DeskDraw(pDisplay, DeskInfo);

        Touch_EmptyFifo(pTouch);

        downscale_matrix(drawing_grayscale_m,
                         &downscaled_grayscale_m,
                         DOWNSCALE_FACTOR);

        matrix_flatten(in, downscaled_grayscale_m, 0);

        GUI_PRINTF("\n");
        GUI_PRINTF("Inference operation start... \n");

        t_inf_0 = times(NULL);

        if (forw_prop(*in, out, nn, relu_funct) == 0){
            GUI_DEBUG_NN_PRINTF("ERROR: Forward propagation
            error \n");
            status = 0;       // Error status
            break;
        }

        char char_out = nn_output_convert(*out);
        t_inf_1 = times(NULL);

        GUI_PRINTF("Inference operation completed after %f4
        seconds \n", (float)(t_inf_1 - t_inf_0)/1000);
        GUI_PRINTF("Output value: %c \n", char_out);

        ...

    }
```

Listing 18: Sending the input to the Neural Network

The `downscale_matrix()` function resizes a 140x140 black and white pixel matrix down to a 28x28 grayscale matrix. Since the 140x140 matrix is exactly 25 times larger than the 28x28 one, it averages the values of the 25 pixels from the larger matrix and saves that grayscale value into the smaller one.
This adapts the data to the neural network's input layer, which consists of 784 neurons (28x28), while keeping the averaged information as grayscale. It's worth noting that an `OFFSET_VALUE` was added for any saved values greater than zero, a neat tweak that further improves how closely it matches the neural network's training dataset.

```
int downscale_matrix(Matrix in, Matrix *out, int
    dwn_fact){

    ...

    float acc = 0;
    unsigned int i,j,k,m;
    for (i = 0; i < DRAW_RECT_WIDTH; i+=dwn_fact){
        for (j = 0; j < DRAW_RECT_HEIGHT; j+=dwn_fact){
            for (k = 0; k < dwn_fact; k++){
                for (m = 0; m < dwn_fact; m++){
                    // Matrix indexing syntax
                    // m[column * row_length + row] ->
                    // m[row][column]
                    acc +=
                    in.data[(j + m) * in.row + (i + k)];
                }
            }

            // The new value is represented with
            // gray scale on 8 bit (256)
            acc = acc / 25;

            // Regulate the offset
            acc =
            (acc > 0.0f) ? (acc + OFFSET_VALUE) : (acc);
            acc =
            (acc > 1.0f) ? (1.0f) : (acc);

            // Matrix indexing syntax
            // m[column * row_length + row] ->
            // m[row][column]
            (*out).data[
            (j / dwn_fact) * (*out).row + (i / dwn_fact)
            ] = acc;

            acc = 0;
        }
    }

    return 1;   // Exit success
}
```

Listing 19: Matrix downscaling

3.2.1 LCD Display Control Library

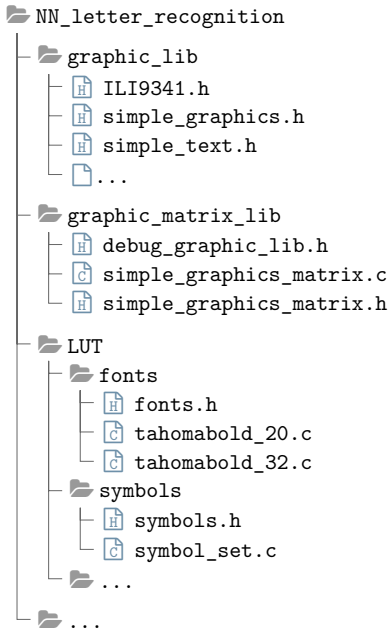


Figure 18: Graphic library function folder tree

To control the display, the `graphic_lib` library has been used; it contains different headers to control the LCD module.
The heart of the library is the `ILI9341.h` header, which contains all the functions required to initialize the LCD and write to it.
Two other headers are also present:
`simple_graphics.h` and `simple_text.h`.
These two headers contain all the functions that, by using the
`ILI9341.h` functions, draw useful geometric shapes such as rectangles, circles, etc.
The functions contained in
`simple_text.h` are used to write letters and numbers using a specified font, saved into a LUT and defined in another file and linked by another header called `fonts.h`.
The `graphic_lib` library files are stored into the folder `./graphic_lib` the fonts LUT and header are store into the folder `./LUT/fonts`.
The header and source files are distributed in different folders, the folder tree is shown in Figure 18.

3.2.2 SPI-Based Touchscreen Control Library

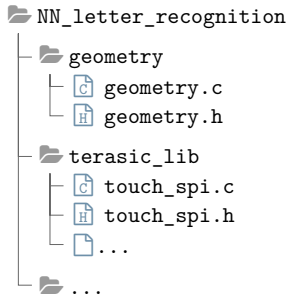


Figure 19: Touch screen library function folder tree

In these library files are present some useful functions to let the touchscreen to be programmed more easily. The `touch_spi.c` contains all those functions to get the position of the pointer on the screen (`Touch_GetXY()`), to sense if the pen is pressed (`Touch_IsPenPressed()`) or to empty the FIFO buffer (`Touch_EmptyFifo()`).
The `geometry.c` contains all the functions to create and handle rectangles to sense a specific area of the screen.

3.3 Main structure

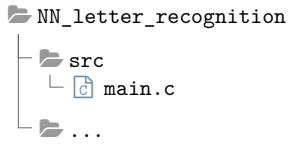


Figure 20: Main location on folder tree

The `main()` function's job is to call the initialization routines for the LT24 touchscreen display, print to the console which dataset was selected, and finally launch the `GUI())` function, which handles the actual user interface and triggers the neural network processing.
The code is shown in Listing 20.

```
int main(void){
    // Init display
    alt_video_display Display = {.interlace = 0,
    .bytes_per_pixel = 2, ... };

    // Init touch
    TOUCH_HANDLE *pTouch;
    pTouch = Touch_Init(TOUCH_PANEL_SPI_BASE, ... );
    if(!pTouch){
        printf("Failed to init touch\r\n");
        return 0;       //Exit failure
    }
    printf("Init touch successfully\r\n");

    // Init LCD
    DESK_INFO DeskInfo; RECT rcTouch;
    GUI_init(&Display, &DeskInfo, &rcTouch);

    // Init neural network
    Matrix in, out;
    NN_f_conn nn;
    int status = NN_Matrix_init(&in, &out, &nn);

    if(status == 1){
        #ifdef MNIST_DATASET
            printf("MNIST DATASET loaded\r\n");
        #else
            printf("EMNIST DATASET loaded\r\n");
        #endif
    }else{
        #ifdef MNIST_DATASET
            printf("Error loading MNIST DATASET\r\n");
        #else
            printf("Error loading EMNIST DATASET\r\n");
        #endif
    };

    if(status == 1)
        // Print GUI
        status = GUI(&Display, pTouch, &in, &out, ... );

    // Deallocate matrix from dynamic memory
    matrix_free(&in); matrix_free(&out); NN_free(&nn);

    return status;       //Exit status
}
```

Listing 20: Main

4 Debug and Verification

The code is composed of different parts working together, and when problems occur, it is important to identify their source in order to correct them.
The main issues are:

- **Logic code errors**
- **Errors in matrix data storage**
- **Inference computation errors**

Different techniques were used to debug the system, as explained below.

4.1 Debug print methodology

To debug the code, the JTAG-UART peripheral was used to print debug messages.
However, UART messages must be written into a buffer, and if too many messages are generated, the buffer can become full, resulting in code slowdown and an excessive amount of printed output.
To avoid this problem, conditional compilation was used to define a `DEBUG_PRINTF()` macro that prints data through the UART only if the constant `DEBUG` is defined; otherwise, the macro does nothing.
An example of this implementation is shown in Listing 21, where `printf()` is the standard C function defined in the `stdio.h` library that prints data through the UART, and `matrix_print()` is a function defined in the `matrix_lib.h` library that internally uses the `printf()` function to print the data contained in a `Matrix` structure. The `Matrix` type is defined in `matrix_lib.h` and described in subsubsection 3.1.1.

```
1   #include <stdio.h>
2   #include "matrix_lib.h"
3
4   // Comment this line to disable debugging messages
5   #define NN_LIB_DEBUG
6
7   #ifdef NN_LIB_DEBUG
8       #define DEBUG_NN_PRINTF(...)     printf(__VA_ARGS__)
9       #define DEBUG_NN_MATRIX_PRINTF(m)   matrix_print(m)
10  #else
11      #define DEBUG_NN_PRINTF(...)
12      #define DEBUG_NN_MATRIX_PRINTF(m)
13  #endif
```

Listing 21: `DEBUG_PRINT()` macro definition for the NN library

4.1.1 Python Matrix visualization

The letters shown on the LCD are stored in a 140×140 matrix.
Each element of the matrix contains the value 1.0 if the corresponding pixel is colored, and 0.0 otherwise.
The matrix is then resized to a smaller 28×28 version.
To verify that the resizing and processing steps work correctly, a debug print was used to display the matrix values, together with a Python script that visualizes the generated data.
In Figure 21, a comparison between a letter written on the LCD and a letter taken from the dataset is shown.

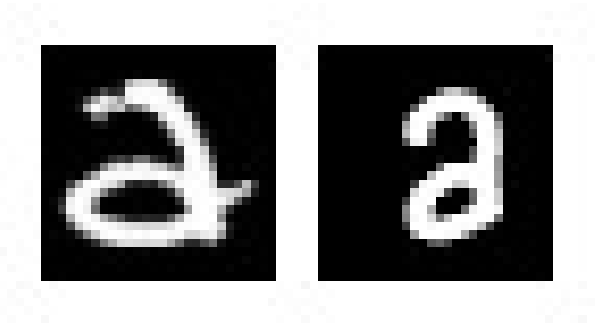


Figure 21: On the left is shown a letter "a" from the dataset, while on the right is shown a letter "a" written manually on the screen

4.2 Python Neural Network confront

The last aspect to verify is the inference computation of the neural network.
To do this, the same network was implemented in Python using the PyTorch library.
The weights and biases were then loaded into the network.
After that, several inputs were processed using both the Python implementation and the network programmed into the FPGA, and the corresponding output values were compared to verify that they match.
In Table 1, we can see the maximum output error obtained when recognizing the letter "a" shown in Figure 21.

Metric	Value
Predicted class	36
Corresponding character	"a"
Max absolute error	$3.8147 \cdot 10^{-6}$

Table 1: Comparison between FPGA and PyTorch inference outputs

4.3 Neural Network accuracy

The neural network training was performed in Python using a library called PyTorch.
Using this library, the neural network was instantiated layer by layer, and the activation functions were also defined.
To perform different tests, two fully connected neural networks with three layers were defined, making them easier to implement using the Neural Network library developed in C for the Nios II processor.
One network was trained to recognize only numbers, while the other was trained to recognize both letters and numbers.
The datasets used were the MNIST dataset for digit recognition and the EMNIST dataset for alphanumeric character recognition.
The code used to instantiate the network in python is shown in Listing 22.

```
    class Net(nn.Module):
        def __init__(self):
            super(Net, self).__init__()
            #input matrix 28x28 flattened
            self.fc1 = nn.Linear(784, 256)
            #hidden layer with 256 inputs and 128 outputs
            self.fc2 = nn.Linear(256, 128)
            #output vector of 10 numbers
            self.fc3 = nn.Linear(128, 10)

        # x represents our data
        def forward(self, x):
            #flattening the 28x28 input matrix
            x = torch.flatten(x, 1)
            #computing the output matrix
            x = F.relu(self.fc1(x))
            x = F.relu(self.fc2(x))
            x = self.fc3(x)
            #return the output matrix
            return x
```

Listing 22: MNIST letter recognition neural network definition in python using PyTorch

With the same method has been defined also the network for EMNIST dataset (for letters and numbers).
The training has been done with a minibatch of 256 elements taken randomly from the dataset; this means that weights and biases of the network are updated using a subset of the dataset to reduce the computational complexity.
Every time that the entire minibatch has been sent to the network and the weights and biases updated, a training epoch is completed.
Every epoch of training, the program do a test with 2000 elements taken randomly from the dataset and test the accuracy of the network and the average

training loss.
The accuracy (Acc) is defined as the number of correct guess of the network over the total, and the average loss ($\mathcal{L}_{\text{avg}}$) is defined as the average difference of the computed output from the ideal output (Equation 6).

$$\begin{aligned} \text{Acc} &= \frac{N_{\text{correct}}}{N} \\ \mathcal{L}_{\text{avg}} &= \frac{1}{N} \sum_{i=1}^{N} \|\vec{a}_i - \vec{y}_i\| \end{aligned} \tag{6}$$

where:

- N: total number of samples
- N_{correct}: number of correctly predicted samples
- $\vec{a}_i$: computed output for sample i
- $\vec{y}_i$: predicted output for sample i

A graph of accuracy and training loss during a 30-epoch training was produced for both models (MNIST and EMNIST), as shown in Figure 22.

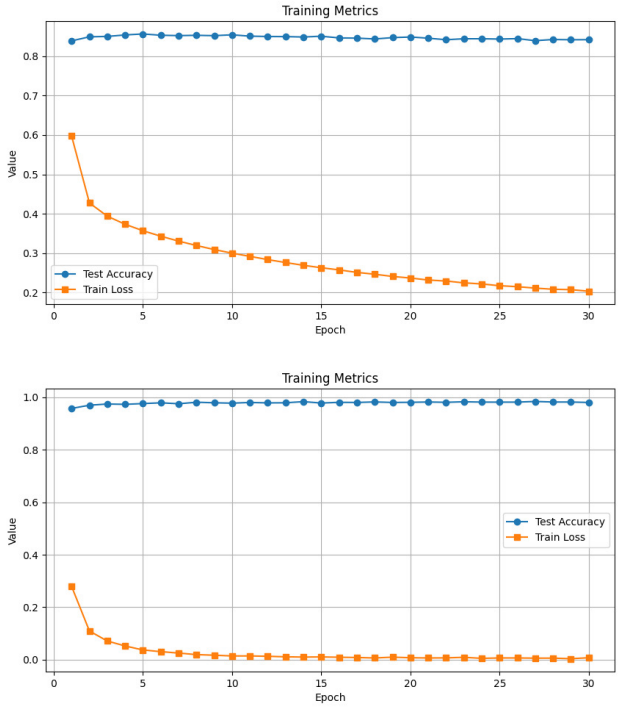


Figure 22: Metrics graph: the bottom graph shows the MNIST dataset, while the upper graph shows the EMNIST dataset

As can be seen from Figure 22, for both networks the accuracy saturates after approximately 5 epochs.
However, for the MNIST dataset, the test accuracy also saturates after approximately 10 epochs, whereas for the EMNIST dataset the test accuracy saturates after approximately 30 epochs.
The weights and biases matrix used for inference inside the niosII has been generated after 30 epochs of training for both datasets.

Note: Even if the accuracy for the EMNIST dataset is approximately 85%*, the dataset contains letters that occupy the entire* 28×28 *matrix, while on the LCD writing area (after downscaling) the letters usually occupy a smaller region.*
For this reason, the training was performed by reducing the size of the dataset letters.
This effect can also be seen in Figure 21 where the letter from the dataset is bigger than the one written manually on the screen.

4.4 Neural Network inference time

Once the neural network had been implemented on the FPGA, two operations were performed to improve the inference time.

- A hardware floating-point operator was instantiated to accelerate, through hardware, the mathematical operations involving weights and biases, mainly additions and multiplications.
- The compiler optimization settings were enabled to optimize the generated code (optimization level 3).

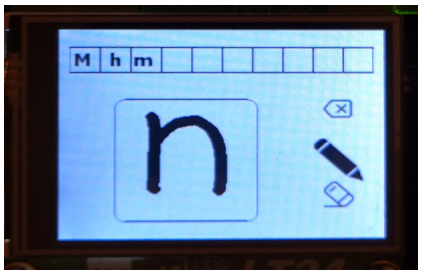
To evaluate the improvement introduced by each modification to the inference time, the inference execution time was measured via software, and the average inference time was analyzed both with and without these optimizations.

Optimization	Inference time
No hardware fp. operator, No compiler optimization	6.101 s
No hardware fp. operator, With compiler optimization	3.150 s
With hardware fp. operator, No compiler optimization	2.089 s
With hardware fp. operator and Compiler optimization	0.509 s

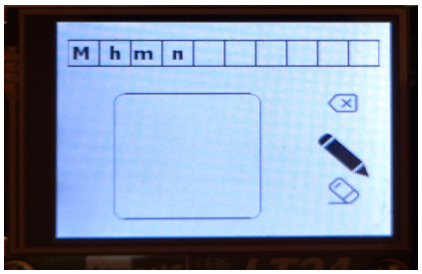
Table 2: Inference Time Dependence on Optimization Methods

4.5 Project Demonstration

The following images show the user drawing a character on the screen and the neural network recognizing the character drawn, then showing it in the letter box area on the top section of the screen.

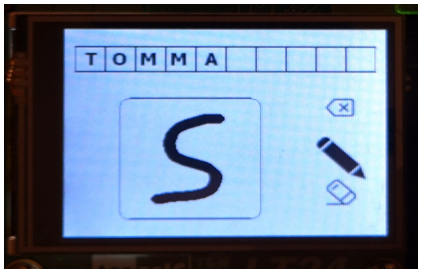


(a) Letter "n" drawn by the user

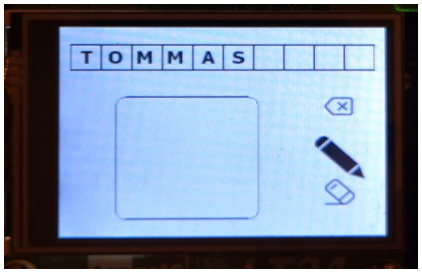


(b) Letter "n" recognized by the NN

Figure 23: Showcasing the recognition of the letter "n"

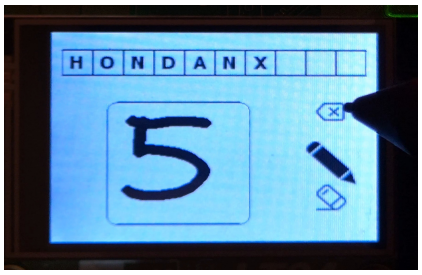


(a) Letter "S" drawn by the user

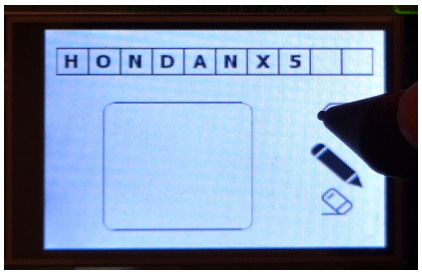


(b) Letter "S" recognized by the NN

Figure 24: Showcasing the recognition of the letter "S"



(a) Number "5" drawn by the user



(b) Number "5" recognized by the NN

Figure 25: Showcasing the recognition of the number "5"

5 Conclusions

The goal of this project was not to achieve a specific level of accuracy with the neural network implemented on the FPGA, but rather to successfully implement the network and obtain an inference time low enough to be unnoticeable to a human user.
This requirement was achieved using a complete software implementation while accelerating the floating-point arithmetic operations through dedicated hardware.
If this implementation had not been sufficient to achieve an adequately low inference time, several improvements could have been introduced.

5.1 Possible upgrades

The possible upgrades are various, but the most important metrics to improve are:

- **Inference Time**: reducing the inference time is important to compute the output faster.
- **Average Accuracy**: improving the accuracy of the network allows more characters to be recognized correctly on average.

Inference time In subsection 4.4, some optimizations and their impact on the inference time of the network are presented.
However, if even with these improvements the inference time is not sufficiently small, an alternative approach would be to describe the entire network in hardware using an HDL language and integrate it as a custom module of the CPU. This method is more complex and less modular than a software-based approach, but it can achieve significantly higher speed.
If even this solution is not sufficient to meet the inference time requirements, dedicated external hardware accelerators should be used to improve multiply-and-accumulate operations (like GPUs).
Another possible approach is to quantize the weights and biases into integer values, simplifying multiplication and addition operations; however, this may lead to reduced accuracy.

Average Accuracy The network average accuracy with MNIST dataset, is approximately the 85%, but this value is computed by using some letters from the dataset; the letters wrote on the screen are usually different in orientation and dimensions with the one on the dataset, so the resulting accuracy is less than the one computed (the 85% accuracy computed during training, see subsection 4.3).
To increase the real accuracy some precaution has been taken, like the one described in the *Note* at the end of subsection 4.3, but the accuracy remains always lower than 85%.
During the test has been tried different types of neural networks, with more

hidden layers, or with less hidden layers but with more neurons, but the accuracy increased to the utmost at 86%, the one used for the project is the best compromise of memory-footprint and accuracy of the network.
To rise the accuracy a different type of network must be used, like convolutional networks or combination of convolutional and fully-connected networks like LeNet-5 (Reference [1]).
Another limitation is that the network performs poorly when recognizing distorted or scaled letters and numbers.
To reduce this problem, the network could be trained using an enlarged dataset generated by applying transformations such as distortion, scaling, and rotation to the original characters.

Note: To compute the real accuracy of the network, different letters would need to be written manually and compared with the expected results.
This process can be performed, but it is time-consuming and was not carried out in this project.

5.2 Implementation on Different FPGA Devices

As can be observed in subsection 2.3, the accelerated floating-point arithmetic project instantiates a significant amount of hardware resources. In particular, the most heavily utilized resources are the embedded 9-bit multipliers: the design instantiates 16 of them.
Another largely utilized resource is the on-chip memory, with a total usage of $180,992$ memory bits.
These resources are commonly available on most modern FPGAs.

- The MAX10 FPGA (the one we used for this project) has 288 available embedded 9bit-multiplier, so their usage for this project is approximately the 5.5%, and $1,677,312$ memory bits, corresponding to a usage of approximately the 10.8%
- The CycloneII, an Altera older FPGA, has 70 available embedded 9bit-multiplier, so their usage for this project is approximately the 22.8%, and $483,840$ memory bits, corresponding to a usage of approximately the 37.4%

The relatively low resource utilization allows the proposed architecture to be implemented even on older FPGA devices.
However, timing analysis may still report violations at the target operating frequency due to technological differences and the lower performance of older FPGA architectures.
This results in a lower maximum operating frequency and, consequently, a longer inference time.
In some cases, especially in the presence of hold-time violations, the architecture may fail to operate correctly even when the clock frequency is reduced.

References

[1] Yann LeCun, Léon Bottou, Yoshua Bengio, Patrick Haffner (1998) *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324. DOI: 10.1109/5.726791.

[2] RocketBoards.org (2024) *24" Touch LCD Painter Demo on Atlas-SoC*. Available at: rocketboards.org

[3] Terasic (2015) *LT24 2.4-inch Touch LCD Module / Atlas-SoC Kit Resources*. Available at: terasic.com.tw

[4] Terasic (2015) *DE10-Lite Development and Education Board*. Available at: terasic.com.tw

[5] PyTorch (2024) *Defining a Neural Network*. Available at: pytorch.org

[6] Liquet, B., Moka, S. and Nazarathy, Y. (2021) *General Fully Connected Neural Networks*. Available at: deeplearningmath.org

[7] Intel Corporation (2026) *USB Blaster III FPGA Development Cable User Guide: Creating JTAG Timing Constraints in Quartus Prime Software*, Altera Documentation, Section 5.4.1. Available at: altera.com.